

```
import pandas as pd
import numpy as np
```

```
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krisha","Pooja"],"Maths_marks":[60,70,80,90,np.nan],"E
```

```
df1=pd.DataFrame(dict1)
```

```
df1.isnull()
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	False	False	False	False	False	False	True	
1	False	False	False	False	False	True	False	
2	False	False	False	False	True	False	False	
3	False	False	False	True	False	False	False	
4	False	False	True	False	False	False	False	

```
df1.isnull().sum()
```

	0
Roll_No	0
Name	0
Maths_marks	1
English_marks	1
Science_marks	1
History_marks	1
Geography_marks	1

dtype: int64

```
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Roll_No                5 non-null     int64
1   Name                   5 non-null     object
2   Maths_marks            4 non-null     float64
3   English_marks          4 non-null     float64
4   Science_marks          4 non-null     float64
5   History_marks          4 non-null     float64
6   Geography_marks        4 non-null     float64
dtypes: float64(5), int64(1), object(1)
memory usage: 412.0+ bytes
```

```
bool_series = pd.notnull(df1)
bool_series
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	True	True	True	True	True	True	False	
1	True	True	True	True	True	False	True	
2	True	True	True	True	False	True	True	
3	True	True	True	False	True	True	True	
4	True	True	False	True	True	True	True	

Next steps: [Generate code with bool_series](#) [New interactive sheet](#)

```
# filling missing value using fillna()
df1.fillna(0,inplace=True)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	90.0	0.0	
1	2	Dipak	70.0	80.0	90.0	0.0	60.0	
2	3	Shreya	80.0	90.0	0.0	60.0	70.0	
3	4	Krishna	90.0	0.0	60.0	70.0	80.0	
4	5	Pooja	0.0	60.0	70.0	80.0	90.0	

Next steps:

Generate code with df1

New interactive sheet

```
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	90.0	NaN	
1	2	Dipak	70.0	80.0	90.0	NaN	60.0	
2	3	Shreya	80.0	90.0	NaN	60.0	70.0	
3	4	Krishna	90.0	NaN	60.0	70.0	80.0	
4	5	Pooja	NaN	60.0	70.0	80.0	90.0	

Next steps:

Generate code with df1

New interactive sheet

```
# filling a missing value with
# previous ones
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krishna","Pooja"],"Maths_marks":[60,70,80,93,np.nan],"E
df1=pd.DataFrame(dict1)
```

```
df1.fillna(method ='pad')
```

```
/tmp/ipykernel_2435/1848718746.py:1: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a
df1.fillna(method ='pad')
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	76.0	60.0	
2	3	Shreya	80.0	40.0	91.0	60.0	70.0	
3	4	Krishna	93.0	40.0	60.0	70.0	80.0	
4	5	Pooja	93.0	60.0	70.0	80.0	90.0	

```
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krishna","Pooja"],"Maths_marks":[60,70,80,93,np.nan],"E
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	NaN	62.0	
2	3	Shreya	80.0	40.0	NaN	68.0	70.0	
3	4	Krishna	93.0	NaN	65.0	70.0	80.0	
4	5	Pooja	NaN	50.0	70.0	80.0	90.0	

Next steps:

Generate code with df1

New interactive sheet

```
# filling a missing value with
# next ones
df1.fillna(method = 'bfill')
```

```
/tmp/ipykernel_2435/3214477383.py:3: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a
df1.fillna(method = 'bfill')
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	62.0	
1	2	Dipak	70.0	80.0	91.0	68.0	62.0	
2	3	Shreya	80.0	40.0	65.0	68.0	70.0	
3	4	Krishna	93.0	50.0	65.0	70.0	80.0	
4	5	Pooja	NaN	50.0	70.0	80.0	90.0	

```
dict1={"Roll_No": [1,2,3,4,5], "Name": ["Amol", "Dipak", "Shreya", "Krisha", "Pooja"], "Maths_marks": [60,70,80,93,np.nan], "E
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	NaN	62.0	
2	3	Shreya	80.0	40.0	NaN	68.0	70.0	
3	4	Krishna	93.0	NaN	65.0	70.0	80.0	
4	5	Pooja	NaN	50.0	70.0	80.0	90.0	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
# filling a null values using fillna()
df1["Maths_marks"] = df1["Maths_marks"].fillna(45)
df1["English_marks"] = df1["English_marks"].fillna(55)
df1["Science_marks"] = df1["Science_marks"].fillna(65)
df1["History_marks"] = df1["History_marks"].fillna(75)
df1["Geography_marks"] = df1["Geography_marks"].fillna(85)
```

df1

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	85.0	
1	2	Dipak	70.0	80.0	91.0	75.0	62.0	
2	3	Shreya	80.0	40.0	65.0	68.0	70.0	
3	4	Krishna	93.0	55.0	65.0	70.0	80.0	
4	5	Pooja	45.0	50.0	70.0	80.0	90.0	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
dict1={"Roll_No": [1,2,3,4,5], "Name": ["Amol", "Dipak", "Shreya", "Krisha", "Pooja"], "Maths_marks": [60,70,80,93,np.nan], "E
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	NaN	62.0	
2	3	Shreya	80.0	40.0	NaN	68.0	70.0	
3	4	Krishna	93.0	NaN	65.0	70.0	80.0	
4	5	Pooja	NaN	50.0	70.0	80.0	90.0	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
# filling a null values using fillna()
df1["Maths_marks"] = df1["Maths_marks"].fillna(int(df1["Maths_marks"].mean()))
```

df1

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	NaN	62.0	
2	3	Shreya	80.0	40.0	NaN	68.0	70.0	
3	4	Krishna	93.0	NaN	65.0	70.0	80.0	
4	5	Pooja	75.0	50.0	70.0	80.0	90.0	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krisha","Pooja"],"Maths_marks":[60,70,80,93,np.nan],"E
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	NaN	62.0	
2	3	Shreya	80.0	40.0	NaN	68.0	70.0	
3	4	Krishna	93.0	NaN	65.0	70.0	80.0	
4	5	Pooja	NaN	50.0	70.0	80.0	90.0	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
# will replace Nan value in dataframe with value 85
df1.replace(to_replace = np.nan, value = 85)
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	85.0	
1	2	Dipak	70.0	80.0	91.0	85.0	62.0	
2	3	Shreya	80.0	40.0	85.0	68.0	70.0	
3	4	Krishna	93.0	85.0	65.0	70.0	80.0	
4	5	Pooja	85.0	50.0	70.0	80.0	90.0	

```
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krisha","Pooja"],"Maths_marks":[60,70,80,93,np.nan],"E
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	NaN	62.0	
2	3	Shreya	80.0	40.0	NaN	68.0	70.0	
3	4	Krishna	93.0	NaN	65.0	70.0	80.0	
4	5	Pooja	NaN	50.0	70.0	80.0	90.0	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
# to interpolate the missing values
df1.interpolate(method='linear', limit_direction='forward')
#c=(a+b)/2
```

```
/tmp/ipykernel_2435/1794567949.py:2: FutureWarning: DataFrame.interpolate with object dtype is deprecated and will raise an error in a future version of pandas.
df1.interpolate(method='linear', limit_direction='forward')
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	72.0	62.0	
2	3	Shreya	80.0	40.0	78.0	68.0	70.0	
3	4	Krishna	93.0	45.0	65.0	70.0	80.0	
4	5	Pooja	93.0	50.0	70.0	80.0	90.0	

```
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krisha","Pooja"],"Maths_marks":[60,70,80,93,75],"English_marks":[70,80,40,45,50],"Science_marks":[80,91,78,65,70],"History_marks":[76,72,68,70,80],"Geography_marks":[NaN,62,70,80,90]}
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60	70.0	80.0	76.0	NaN	
1	2	Dipak	70	80.0	91.0	NaN	62.0	
2	3	Shreya	80	40.0	NaN	68.0	70.0	
3	4	Krishna	93	NaN	65.0	70.0	80.0	
4	5	Pooja	75	50.0	70.0	80.0	90.0	

Next steps:

Generate code with df1

New interactive sheet

```
# using dropna() function
df1.dropna()
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
4	5	Pooja	75	50.0	70.0	80.0	90.0	

```
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krisha","Pooja"],"Maths_marks":[60,70,80,93,np.nan],"English_marks":[70,80,40,45,50],"Science_marks":[80,91,78,65,70],"History_marks":[76,72,68,70,80],"Geography_marks":[NaN,62,70,80,90]}
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	NaN	62.0	
2	3	Shreya	80.0	40.0	NaN	68.0	70.0	
3	4	Krishna	93.0	NaN	65.0	70.0	80.0	
4	5	Pooja	NaN	50.0	70.0	80.0	90.0	

Next steps:

Generate code with df1

New interactive sheet

```
# using dropna() function
df1.dropna(how = 'all',inplace=True)
```

```
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krisha","Pooja"],"Maths_marks":[60,70,80,93,np.nan],"English_marks":[70,80,40,45,50],"Science_marks":[80,91,78,65,70],"History_marks":[76,72,68,70,80],"Geography_marks":[NaN,62,70,80,90]}
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	NaN	62.0	
2	3	Shreya	80.0	40.0	NaN	68.0	70.0	
3	4	Krishna	93.0	NaN	65.0	70.0	80.0	
4	5	Pooja	NaN	50.0	70.0	80.0	90.0	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
# using dropna() function
df1.dropna(axis = 1)
```

	Roll_No	Name	
0	1	Amol	
1	2	Dipak	
2	3	Shreya	
3	4	Krishna	
4	5	Pooja	

```
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krisha","Pooja"],"Maths_marks":[60,70,80,93,np.nan],"E
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	60	
1	2	Dipak	70.0	80.0	91.0	NaN	62	
2	3	Shreya	80.0	40.0	NaN	68.0	70	
3	4	Krishna	93.0	NaN	65.0	70.0	80	
4	5	Pooja	NaN	50.0	70.0	80.0	90	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
# making new data frame with dropped NA values
df2 = df1.dropna(axis = 0, how ='any')
```

df2

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	60	

```
dict1={"Roll_No":[1,2,3,4,5],"Name":["Amol","Dipak","Shreya","Krisha","Pooja"],"Maths_marks":[60,70,80,93,np.nan],"E
df1=pd.DataFrame(dict1)
df1
```

	Roll_No	Name	Maths_marks	English_marks	Science_marks	History_marks	Geography_marks	
0	1	Amol	60.0	70.0	80.0	76.0	NaN	
1	2	Dipak	70.0	80.0	91.0	NaN	62.0	
2	3	Shreya	80.0	40.0	NaN	68.0	70.0	
3	4	Krishna	93.0	NaN	65.0	70.0	80.0	
4	5	Pooja	NaN	50.0	70.0	80.0	90.0	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

OUTLIERS

```
# Importing
import sklearn
from sklearn.datasets import fetch_california_housing
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset (using California housing as an alternative)
```

```
housing = fetch_california_housing()

# Create the dataframe
column_name = housing.feature_names
df_boston = pd.DataFrame(housing.data)
df_boston.columns = column_name
df_boston.head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25

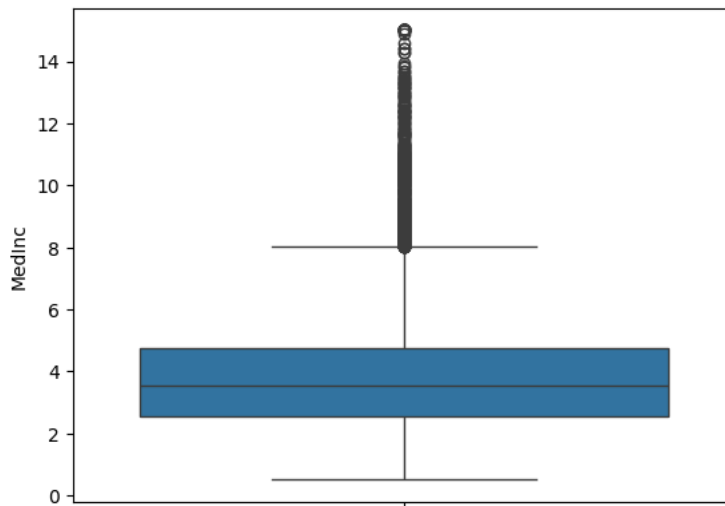
Next steps:

[Generate code with df_boston](#)[New interactive sheet](#)

1. Visualization

```
# Box Plot
import seaborn as sns
sns.boxplot(df_boston['MedInc'])
```

<Axes: ylabel='MedInc'>



```
# Position of the Outlier
print(np.where(df_boston['MedInc']>10))
```

```
(array([ 131,  409,  510,  511,  512,  514,  977,  986, 1563,
        1564, 1566, 1582, 1583, 1586, 1591, 1593, 1602, 1617,
        1621, 1636, 1637, 1644, 1645, 1646, 2213, 2826, 2969,
        3486, 3533, 3858, 4014, 4018, 4042, 4044, 4045, 4046,
        4047, 4049, 4050, 4056, 4070, 4074, 4111, 4116, 4220,
        4229, 4319, 4345, 4352, 4492, 4604, 4605, 4606, 4622,
        4626, 4677, 4678, 5242, 5243, 5244, 5247, 5248, 5250,
        5252, 5253, 5254, 5256, 5257, 5258, 5259, 5260, 5264,
        5269, 5271, 5272, 5275, 5277, 5281, 5291, 5292, 5304,
        5318, 5352, 5360, 5361, 5368, 5494, 5728, 6073, 6204,
        6391, 6399, 6551, 6590, 6591, 6592, 6712, 6720, 6724,
        6725, 6726, 6727, 6728, 6731, 6735, 6736, 6915, 6919,
        8311, 8768, 8790, 8792, 8793, 8794, 8795, 8796, 8797,
        8799, 8805, 8806, 8816, 8817, 8846, 8847, 8848, 8849,
        8850, 8851, 8852, 8853, 8854, 8875, 8876, 8878, 8882,
        9011, 9136, 9316, 9354, 9356, 9370, 9381, 9382, 9418,
        9794, 9811, 10155, 10317, 10318, 10332, 10376, 10401, 10402,
        10405, 10417, 10458, 10464, 10483, 10484, 10517, 10519, 10527,
        10562, 10644, 10661, 10667, 10670, 10671, 10673, 10682, 10724,
        10726, 10727, 10742, 10743, 10744, 10745, 10749, 10757, 10759,
        10760, 10763, 10999, 11000, 11007, 11008, 11010, 11020, 11037,
        11038, 11041, 11469, 11490, 11520, 11521, 11704, 11828, 11912,
        12430, 12948, 13742, 14459, 14467, 14469, 14482, 14486, 14491,
```

```
14686, 15152, 15230, 15231, 15232, 15233, 15235, 15241, 15691,
15693, 15694, 15697, 15698, 15996, 16011, 16019, 16020, 16137,
16171, 16175, 16304, 16828, 16908, 16909, 16910, 16911, 16912,
16913, 16914, 16915, 16918, 16938, 17107, 17108, 17109, 17110,
17111, 17113, 17118, 17158, 17160, 17161, 17162, 17166, 17167,
17170, 17171, 17173, 17310, 17798, 17854, 17855, 17858, 18043,
18052, 18053, 18055, 18059, 18061, 18065, 18066, 18067, 18069,
18078, 18079, 18080, 18081, 18087, 18269, 18280, 18281, 18282,
18283, 18287, 18298, 18325, 18327, 18341, 18347, 18348, 18351,
18352, 18353, 18354, 18355, 18356, 18360, 18361, 18362, 18363,
18366, 18501, 18504, 18619, 19006, 20163, 20376, 20380, 20389,
20426, 20436]),)
```

```
df_boston.info()
```

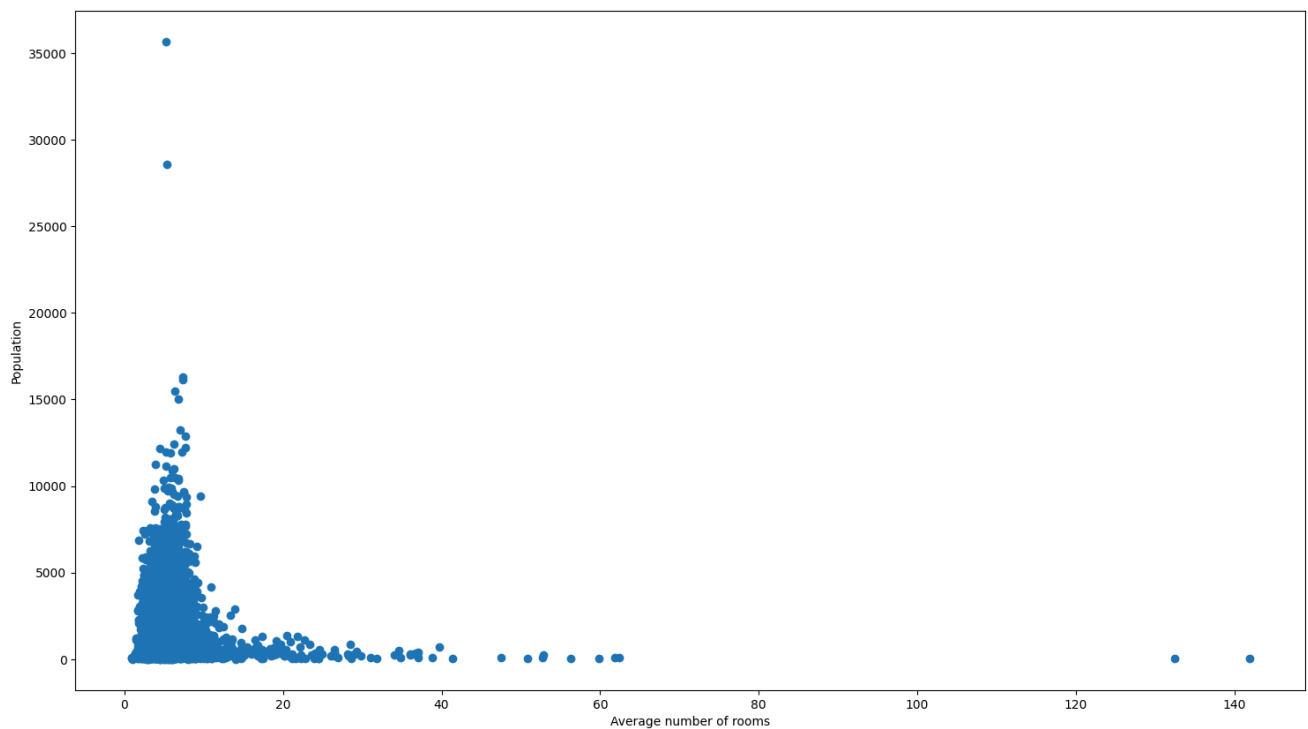
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   MedInc      20640 non-null  float64
1   HouseAge    20640 non-null  float64
2   AveRooms    20640 non-null  float64
3   AveBedrms   20640 non-null  float64
4   Population  20640 non-null  float64
5   AveOccup    20640 non-null  float64
6   Latitude    20640 non-null  float64
7   Longitude   20640 non-null  float64
dtypes: float64(8)
memory usage: 1.3 MB
```

✓ Using ScatterPlot

```
# Scatter plot
fig, ax = plt.subplots(figsize = (18,10))
ax.scatter(df_boston['AveRooms'], df_boston['Population'])

# x-axis label
ax.set_xlabel('Average number of rooms')

# y-axis label
ax.set_ylabel('Population')
plt.show()
```



```
# Position of the Outlier
print(np.where((df_boston['AveRooms']>10) & (df_boston['Population']>20000)))

(array([], dtype=int64),)
```

2. Z-score

```
# Z score
from scipy import stats
import numpy as np

z = np.abs(stats.zscore(df_boston['MedInc']))
print(z)

[2.34476576 2.33223796 1.7826994 ... 1.14259331 1.05458292 0.78012947]
```

```
threshold = 3
```

```
# Position of the outlier
print(np.where(z > 3))
```

```
(array([ 131,  409,  510,  511,  512,  514,  923,  977,  986,
        1541, 1561, 1563, 1564, 1566, 1574, 1582, 1583, 1586,
        1591, 1593, 1602, 1617, 1621, 1636, 1637, 1644, 1645,
        1646, 2213, 2826, 2969, 3486, 3533, 3542, 3858, 3953,
        4014, 4018, 4042, 4044, 4045, 4046, 4047, 4049, 4050,
        4056, 4070, 4074, 4111, 4116, 4220, 4229, 4319, 4345,
        4352, 4492, 4604, 4605, 4606, 4622, 4626, 4677, 4678,
        5242, 5243, 5244, 5246, 5247, 5248, 5250, 5252, 5253,
```

```

5254, 5256, 5257, 5258, 5259, 5260, 5264, 5269, 5271,
5272, 5275, 5277, 5281, 5291, 5292, 5304, 5318, 5352,
5360, 5361, 5368, 5493, 5494, 5728, 5738, 6073, 6204,
6226, 6389, 6391, 6399, 6551, 6576, 6590, 6591, 6592,
6712, 6720, 6724, 6725, 6726, 6727, 6728, 6731, 6732,
6735, 6736, 6915, 6919, 8311, 8768, 8790, 8792, 8793,
8794, 8795, 8796, 8797, 8799, 8800, 8805, 8806, 8809,
8814, 8816, 8817, 8846, 8847, 8848, 8849, 8850, 8851,
8852, 8853, 8854, 8875, 8876, 8878, 8882, 9010, 9011,
9136, 9316, 9354, 9356, 9370, 9381, 9382, 9418, 9794,
9811, 10155, 10166, 10169, 10317, 10318, 10324, 10332, 10376,
10389, 10401, 10402, 10405, 10417, 10458, 10464, 10472, 10483,
10484, 10517, 10519, 10527, 10562, 10644, 10661, 10667, 10670,
10671, 10673, 10682, 10724, 10725, 10726, 10727, 10742, 10743,
10744, 10745, 10749, 10757, 10759, 10760, 10761, 10763, 10839,
10999, 11000, 11007, 11008, 11010, 11020, 11037, 11038, 11041,
11042, 11469, 11490, 11518, 11519, 11520, 11521, 11704, 11828,
11912, 12430, 12948, 13742, 14459, 14467, 14469, 14482, 14486,
14491, 14686, 14718, 15152, 15230, 15231, 15232, 15233, 15235,
15241, 15691, 15693, 15694, 15697, 15698, 15996, 16011, 16019,
16020, 16137, 16171, 16175, 16304, 16828, 16908, 16909, 16910,
16911, 16912, 16913, 16914, 16915, 16918, 16938, 17106, 17107,
17108, 17109, 17110, 17111, 17112, 17113, 17118, 17157, 17158,
17160, 17161, 17162, 17166, 17167, 17170, 17171, 17173, 17310,
17493, 17494, 17798, 17854, 17855, 17858, 18043, 18052, 18053,
18055, 18059, 18061, 18065, 18066, 18067, 18069, 18076, 18078,
18079, 18080, 18081, 18087, 18269, 18277, 18280, 18281, 18282,
18283, 18287, 18298, 18325, 18327, 18341, 18347, 18348, 18351,
18352, 18353, 18354, 18355, 18356, 18360, 18361, 18362, 18363,
18366, 18501, 18504, 18505, 18619, 19006, 20163, 20376, 20380,
20389, 20426, 20436]),)

```

3. IQR (Inter Quartile Range)

```

# IQR
Q1 = np.percentile(df_boston['MedInc'], 25,
                    interpolation = 'midpoint')

Q3 = np.percentile(df_boston['MedInc'], 75,
                    interpolation = 'midpoint')

IQR = Q3 - Q1

```

```

# Above Upper bound
upper = df_boston['MedInc'] >= (Q3+1.5*IQR)

print("Upper bound:", upper)
print(np.where(upper))

# Below Lower bound
lower = df_boston['MedInc'] <= (Q1-1.5*IQR)
print("Lower bound:", lower)
print(np.where(lower))

```

```

16911, 16912, 16919, 16920, 16933, 16937, 16971, 16973, 16984,
16828, 16893, 16908, 16909, 16910, 16911, 16912, 16913, 16914,
16915, 16917, 16918, 16938, 16940, 16990, 16995, 16999, 17021,
17023, 17045, 17106, 17107, 17108, 17109, 17110, 17111, 17112,
17113, 17118, 17138, 17145, 17146, 17147, 17153, 17156, 17157,
17158, 17159, 17160, 17161, 17162, 17163, 17165, 17166, 17167,
17170, 17171, 17173, 17183, 17234, 17289, 17294, 17295, 17310,
17493, 17494, 17717, 17798, 17839, 17854, 17855, 17858, 18038,
18042, 18043, 18046, 18051, 18052, 18053, 18055, 18056, 18058,
18059, 18060, 18061, 18065, 18066, 18067, 18068, 18069, 18071,
18073, 18076, 18077, 18078, 18079, 18080, 18081, 18083, 18085,
18086, 18087, 18088, 18106, 18129, 18155, 18177, 18269, 18270,
18273, 18274, 18277, 18278, 18280, 18281, 18282, 18283, 18285,
18286, 18287, 18291, 18296, 18297, 18298, 18323, 18325, 18327,
18330, 18340, 18341, 18347, 18348, 18351, 18352, 18353, 18354,
18355, 18356, 18357, 18358, 18360, 18361, 18362, 18363, 18364,
18366, 18376, 18377, 18378, 18380, 18381, 18384, 18387, 18388,
18389, 18501, 18504, 18505, 18619, 19006, 19412, 19865, 20163,
20335, 20367, 20375, 20376, 20377, 20380, 20382, 20389, 20421,
20426, 20427, 20428, 20436, 20503]),)
Lower bound: 0      False
1      False
2      False
3      False
4      False
...
20635      False
20636      False
20637      False
20638      False
20639      False
Name: MedInc, Length: 20640, dtype: bool
(array([1, dtype=int64],))

```

✧ Removing the outliers

```

# Importing
import sklearn
from sklearn.datasets import fetch_california_housing
import pandas as pd

# Load the dataset
housing = fetch_california_housing()

# Create the dataframe
column_name = housing.feature_names
df_boston = pd.DataFrame(housing.data)
df_boston.columns = column_name
df_boston.head()

''' Detection '''
# IQR
Q1 = np.percentile(df_boston['MedInc'], 25,
                    interpolation = 'midpoint')

Q3 = np.percentile(df_boston['MedInc'], 75,
                    interpolation = 'midpoint')
IQR = Q3 - Q1

print("Old Shape: ", df_boston.shape)

# Upper bound
upper = np.where(df_boston['MedInc'] >= (Q3+1.5*IQR))
# Lower bound
lower = np.where(df_boston['MedInc'] <= (Q1-1.5*IQR))

''' Removing the Outliers '''
df_boston.drop(upper[0], inplace = True)
df_boston.drop(lower[0], inplace = True)

print("New Shape: ", df_boston.shape)

Old Shape: (20640, 8)
New Shape: (19960, 8)

```

✧ data transformations

```
# importing pandas as pd
import pandas as pd

# Creating the DataFrame
df = pd.DataFrame({"A": [12, 4, 5, None, 1],
                  "B": [7, 2, 54, 3, None],
                  "C": [20, 16, 11, 3, 8],
                  "D": [14, 3, None, 2, 6]})

# Create the index
index_ = ['Row_1', 'Row_2', 'Row_3', 'Row_4', 'Row_5']

# Set the index
df.index = index_

# Print the DataFrame
print(df)
```

	A	B	C	D
Row_1	12.0	7.0	20	14.0
Row_2	4.0	2.0	16	3.0
Row_3	5.0	54.0	11	NaN
Row_4	NaN	3.0	3	2.0
Row_5	1.0	NaN	8	6.0

```
# pass a list of functions
result = df.transform(func = ['sqrt', 'exp'])
result
# Print the result
#print(result)
```

	A		B		C		D	
	sqrt	exp	sqrt	exp	sqrt	exp	sqrt	exp
Row_1	3.464102	162754.791419	2.645751	1.096633e+03	4.472136	4.851652e+08	3.741657	1.202604e+06
Row_2	2.000000	54.598150	1.414214	7.389056e+00	4.000000	8.886111e+06	1.732051	2.008554e+01
Row_3	2.236068	148.413159	7.348469	2.830753e+23	3.316625	5.987414e+04	NaN	NaN
Row_4	NaN	NaN	1.732051	2.008554e+01	1.732051	2.008554e+01	1.414214	7.389056e+00
Row_5	1.000000	2.718282	NaN	NaN	2.828427	2.980958e+03	2.449490	4.034288e+02

Next steps:

[Generate code with result](#)[New interactive sheet](#)

SCALING

Method 1: Using Pandas and Numpy

```
import seaborn as sns
import pandas as pd
import numpy as np

data = sns.load_dataset('iris')
print('Original Dataset')
data.head()
```

Original Dataset

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Next steps:

[Generate code with data](#)[New interactive sheet](#)

```
# Min-Max Normalization
df = data.drop('species', axis=1)
df_norm = (df-df.min())/(df.max()-df.min())
df_norm = pd.concat((df_norm, data.species))
```

```
print("Scaled Dataset Using Pandas")
df_norm.head()
```

Scaled Dataset Using Pandas

	sepal_length	sepal_width	petal_length	petal_width	species
0	0.222222	0.625000	0.067797	0.041667	NaN
1	0.166667	0.416667	0.067797	0.041667	NaN
2	0.111111	0.500000	0.050847	0.041667	NaN
3	0.083333	0.458333	0.084746	0.041667	NaN
4	0.194444	0.666667	0.067797	0.041667	NaN

Next steps:

[Generate code with df_norm](#)[New interactive sheet](#)

```
# skewness along the index axis
df_norm.skew(axis = 0, skipna = True, numeric_only=True)
```

```
0
sepal_length    0.314911
sepal_width     0.318966
petal_length    -0.274884
petal_width     -0.102967
```

dtype: float64

```
# find skewness in each row
df.skew(axis = 1, skipna = True)
```

```

      0
0    0.189328
1    0.404389
2    0.208782
3    0.120130
4    0.090753
...    ...
145   0.351514
146   0.256478
147   0.134795
148  -0.146244
149  -0.185387
150 rows × 1 columns

dtype: float64

```

For a unimodal distribution, negative skew commonly indicates that the tail is on the left side of the distribution, and positive skew indicates that the tail is on the right. In cases where one tail is long but the other tail is fat, skewness does not obey a simple rule. For example, a zero value means that the tails on both sides of the mean balance out overall; this is the case for a symmetric distribution, but can also be true for an asymmetric distribution where one tail is long and thin, and the other is short but fat.

▼ Distribution

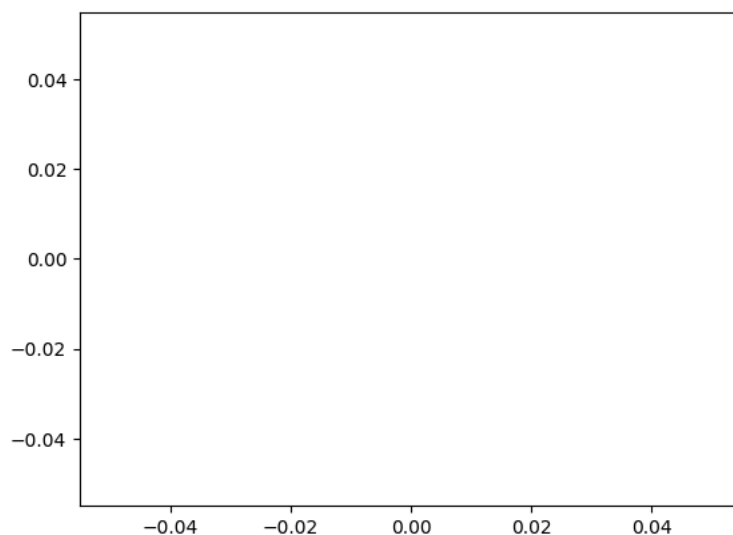
```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
import statistics

# Calculating mean and standard deviation
mean = statistics.mean(df_norm["sepal_length"])
sd = df_norm["sepal_length"].std()

plt.plot(df_norm["sepal_length"], norm.pdf(df_norm["sepal_length"], mean, sd))
plt.show()

```

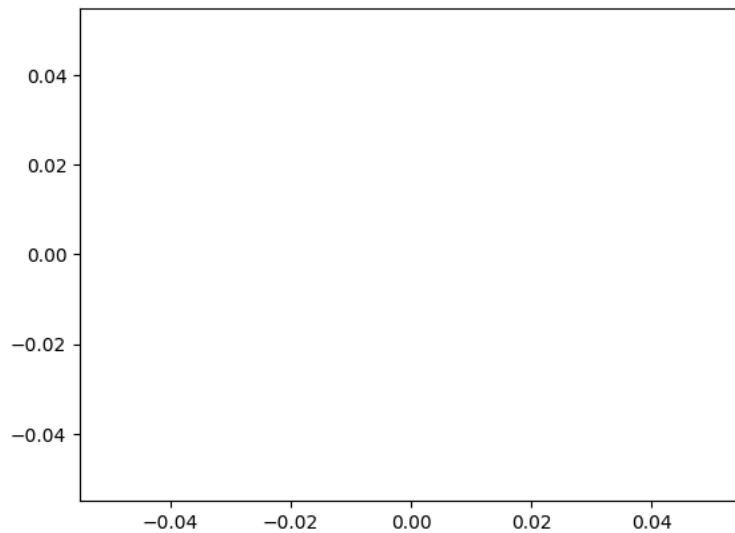


```

# Calculating mean and standard deviation
mean = statistics.mean(df_norm["petal_width"])
sd = df_norm["sepal_length"].std()

```

```
plt.plot(df_norm["petal_width"], norm.pdf(df_norm["petal_width"], mean, sd))
plt.show()
```



```
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

data = sns.load_dataset('iris')
print('Original Dataset')
data.head()

scaler = MinMaxScaler()

df_scaled = scaler.fit_transform(df.to_numpy())
df_scaled = pd.DataFrame(df_scaled, columns=[
    'sepal_length', 'sepal_width', 'petal_length', 'petal_width'])

print("Scaled Dataset Using MinMaxScaler")
df_scaled.head()
```

Original Dataset

Scaled Dataset Using MinMaxScaler

	sepal_length	sepal_width	petal_length	petal_width	
0	0.222222	0.625000	0.067797	0.041667	
1	0.166667	0.416667	0.067797	0.041667	
2	0.111111	0.500000	0.050847	0.041667	
3	0.083333	0.458333	0.084746	0.041667	
4	0.194444	0.666667	0.067797	0.041667	

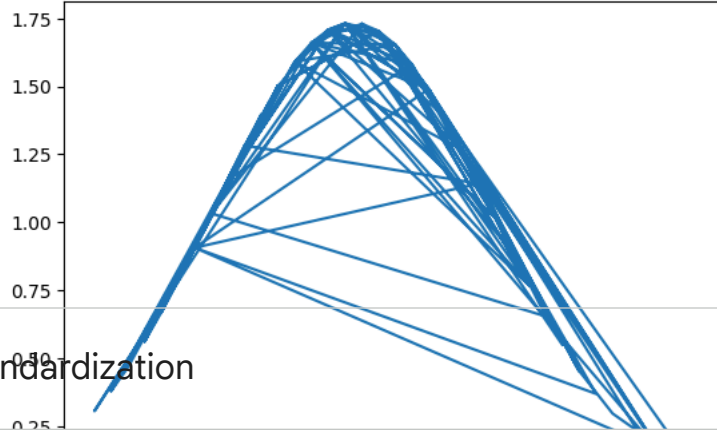
Next steps:

[Generate code with df_scaled](#)

[New interactive sheet](#)

```
# Calculating mean and standard deviation
mean = statistics.mean(df_scaled["sepal_length"])
sd = statistics.stdev(df_scaled["sepal_length"])

plt.plot(df_scaled["sepal_length"], norm.pdf(df_scaled["sepal_length"], mean, sd))
plt.show()
```



Standardization

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
import seaborn as sns

data = sns.load_dataset('iris')
print('Original Dataset')
data.head()
```

Original Dataset

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Next steps: [Generate code with data](#) [New interactive sheet](#)

```
data.drop(["species"], inplace=True, axis=1)
```

data

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2